

Connection to Databricks from Azure Storage (ADLS Gen2) via Access Key

Azure Key Vault + Databricks Secret Scope + ABFSS

Objective: securely connect Azure Databricks to the ADLS Gen2 storage account evanpunnen using its Storage Account Access Key, stored in Azure Key Vault, and validate the connection by reading AdventureWorks_Customers.csv.

Environment used

- Storage account: evanpunnen
- Container: source
- File: AdventureWorks_Customers.csv
- Key Vault: evans-keyvault
- Key Vault secret: evandb8989
- Databricks secret scope: evans-keyvault

Security: never commit the actual Storage Account Access Key or any Key Vault secret value to GitHub. One source screenshot containing a real access key was intentionally excluded from this Git-safe guide.

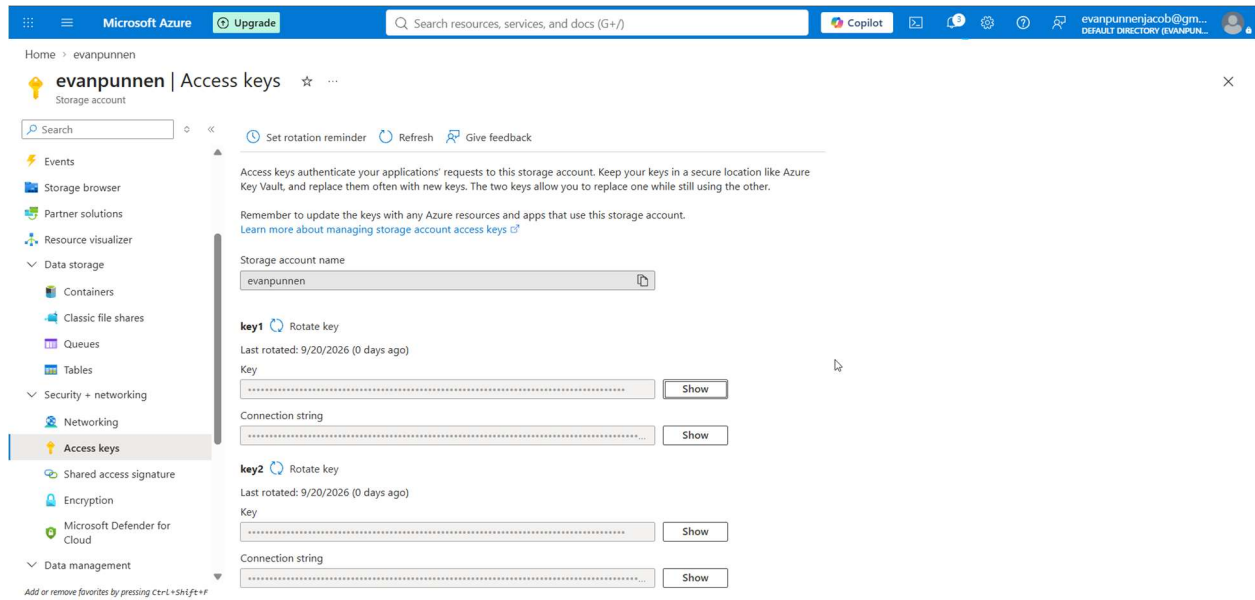
Beginner note: Follow the steps in order. Resource names below match the example environment used in this document. If you use different names, replace them consistently in the later steps and code.

Step 1. Open Azure Databricks and create a Python notebook

Open your Azure Databricks workspace. Click + New → Notebook, choose Python, and create/open the notebook where the connection code will run. Databricks documents + New → Notebook as the standard UI flow for creating a notebook.

Step 2. Get the Storage Account Access Key

Open Azure Portal → Storage accounts → evanpunnen → Security + networking → Access keys. Copy Key 1 and keep it private. Do not paste the key into Git or share it.



1. Do not paste the real key into the notebook, README, screenshots, GitHub, or chat.
2. Copy the access key and keep it private. This value will be stored in Azure Key Vault.
3. Locate Key 1.
4. In the left menu, open Security + networking → Access keys.
5. Open the storage account: evanpunnen.
6. Go to Storage accounts.
7. Open Azure Portal.

Reference screenshot: Storage Account → Access keys.

Step 3. Create the Azure Key Vault

Create/open the Key Vault used to store the Storage Account Access Key. In this setup it is evans-keyvault.

Microsoft Azure Upgrade

Search resources, services, and docs (G+/)

Copilot

evanpunnenjacob@gm...
DEFAULT DIRECTORY (EVANPUN...

Home > Key vaults

Create a key vault

Basics Access configuration Networking Tags Review + create

Azure Key Vault is a cloud service used to manage keys, secrets, and certificates. Key Vault eliminates the need for developers to store security information in their code. It allows you to centralize the storage of your application secrets which greatly reduces the chances that secrets may be leaked. Key Vault also allows you to securely store secrets and keys backed by Hardware Security Modules or HSMs. The HSMs used are Federal Information Processing Standards (FIPS) 140-3 Level 3 validated. In addition, key vault provides logs of all access and usage attempts of your secrets so you have a complete audit trail for compliance.

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * Azure subscription 1

Resource group * Evan_DE
[Create new](#)

Instance details

Key vault name * evan-keyvault

Region * East US

Previous Next Review + create

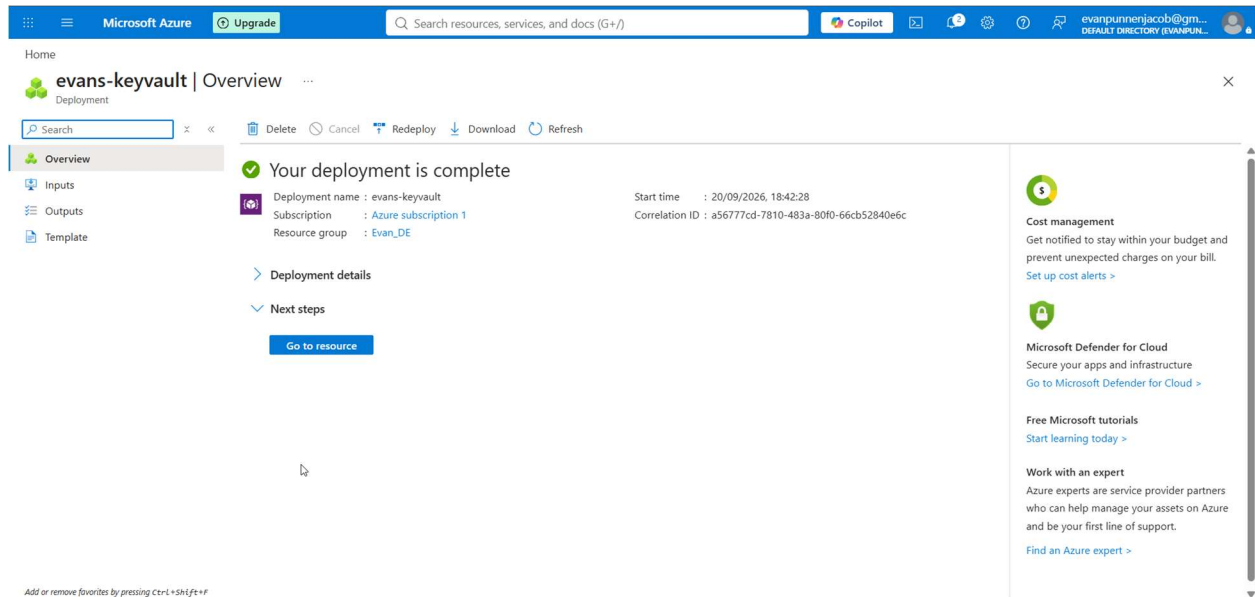
[Give feedback](#)

1. Wait for the deployment to finish.
2. Review the configuration and select Create.
3. Keep the remaining settings appropriate for your Azure environment.
4. Enter the Key Vault name: evans-keyvault.
5. Select the same subscription and resource group used for the project.
6. Select Create.
7. In Azure Portal, search for Key vaults.

Reference screenshot: Create a key vault.

Step 4. Verify the Key Vault deployment

After deployment, open the Key Vault and confirm the deployment is complete.

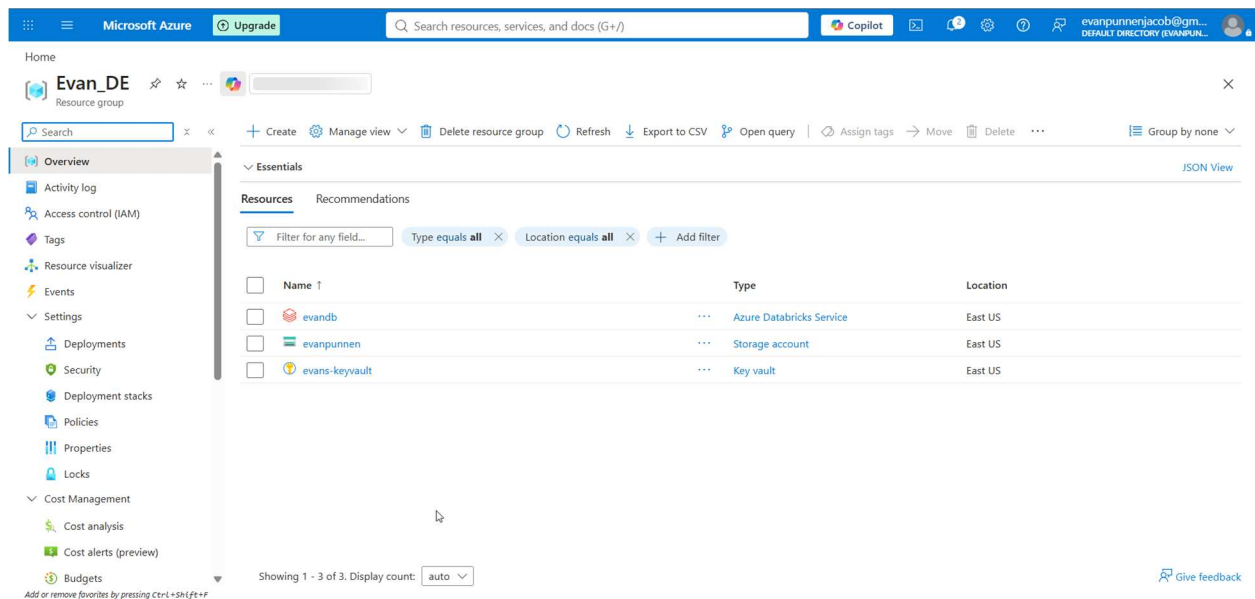


1. Continue only after the Key Vault resource is available.
2. Open evans-keyvault.
3. Confirm that the deployment has completed successfully.
4. Open the deployment notification or go to the Key Vault resource.

Reference screenshot: Key Vault deployment complete.

Step 5. Verify the Azure resources

Open the resource group and confirm the required resources are present: Azure Databricks, the storage account, and the Key Vault.



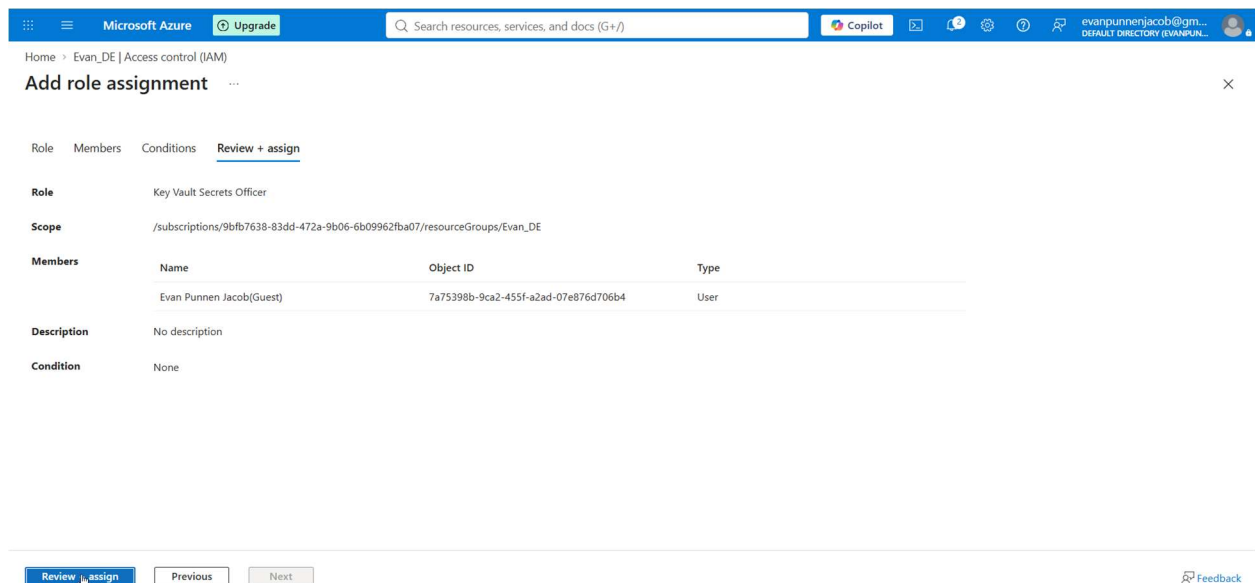
1. If one of these resources is missing, stop and correct the Azure setup before continuing.

2. Confirm that the Key Vault evans-keyvault is present.
3. Confirm that the storage account evanpunnen is present.
4. Confirm that the Azure Databricks workspace is present.
5. Open the resource group used for the project.

Reference screenshot: resource group resources.

Step 6. Configure Key Vault permissions

Configure the required Key Vault secret permissions for the relevant identity. The source screenshot shows a Key Vault Secrets Officer role assignment.

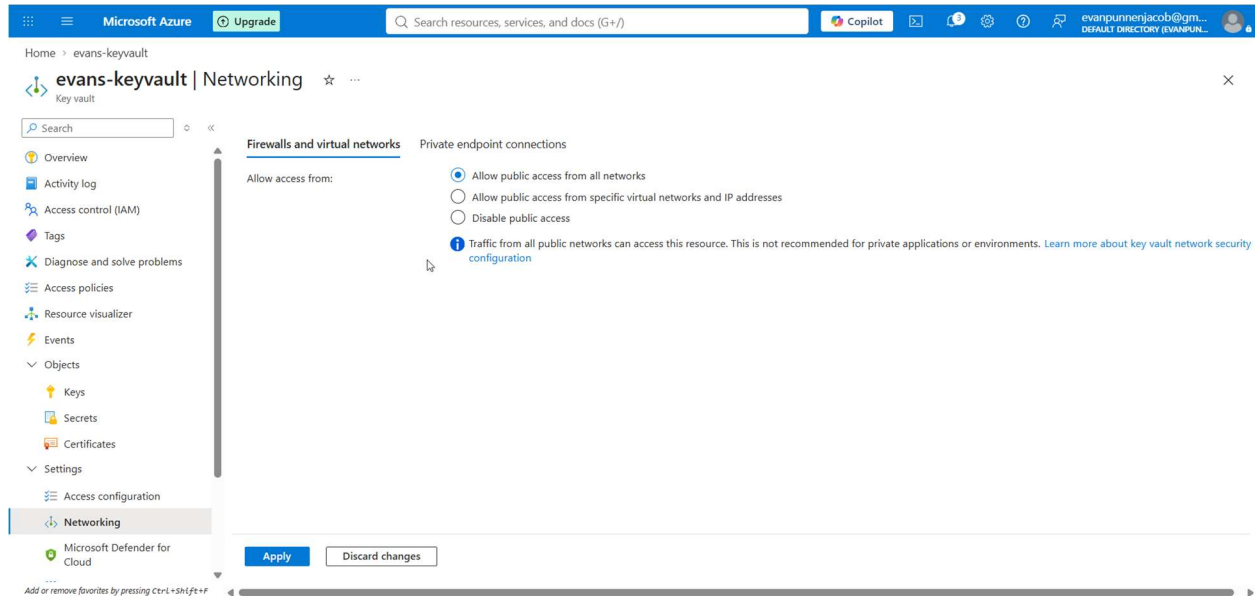


1. Wait briefly for the permission change to propagate before testing access.
2. Review the assignment and complete it.
3. Select the appropriate member/identity for your own Azure environment.
4. The source setup uses the Key Vault Secrets Officer role for the user at the relevant scope.
5. Add the required Key Vault secret-management permission for the identity that will create/manage the secret.
6. Open the role-assignment area.
7. Open Access control (IAM).
8. Open evans-keyvault in Azure Portal.

Reference screenshot: Key Vault Secrets Officer role assignment.

Step 7. Configure Key Vault networking

Configure Key Vault networking according to the access required by your environment. The source screenshot shows public access configuration.



1. Save the networking configuration.
2. The source setup shows public access enabled. If your organization's network policy is different, follow that policy instead.
3. Configure access so the Azure Databricks environment can reach the Key Vault.
4. Review the Public access/networking configuration.
5. Go to Networking.
6. Open evans-keyvault.

Reference screenshot: Key Vault networking.

Step 8. Create the Key Vault secret

Create a secret named evandb8989. The secret VALUE must be the actual Storage Account Access Key from evanpunnen. The secret name is not the access key.

The screenshot shows the 'Create a secret' page in the Microsoft Azure portal. The breadcrumb navigation at the top indicates the path: Home > evans-keyvault | Secrets. The page title is 'Create a secret'. The form includes the following fields and options:

- Upload options:** A dropdown menu set to 'Manual'.
- Name:** A text field containing 'evandb8989' with a green checkmark icon.
- Secret value:** A text field containing masked characters (dots) with a green checkmark icon.
- Content type (optional):** An empty text field.
- Set activation date:** A checkbox that is unchecked.
- Set expiration date:** A checkbox that is unchecked.
- Enabled:** A toggle switch set to 'Yes'.
- Tags:** A link that says '0 tags'.

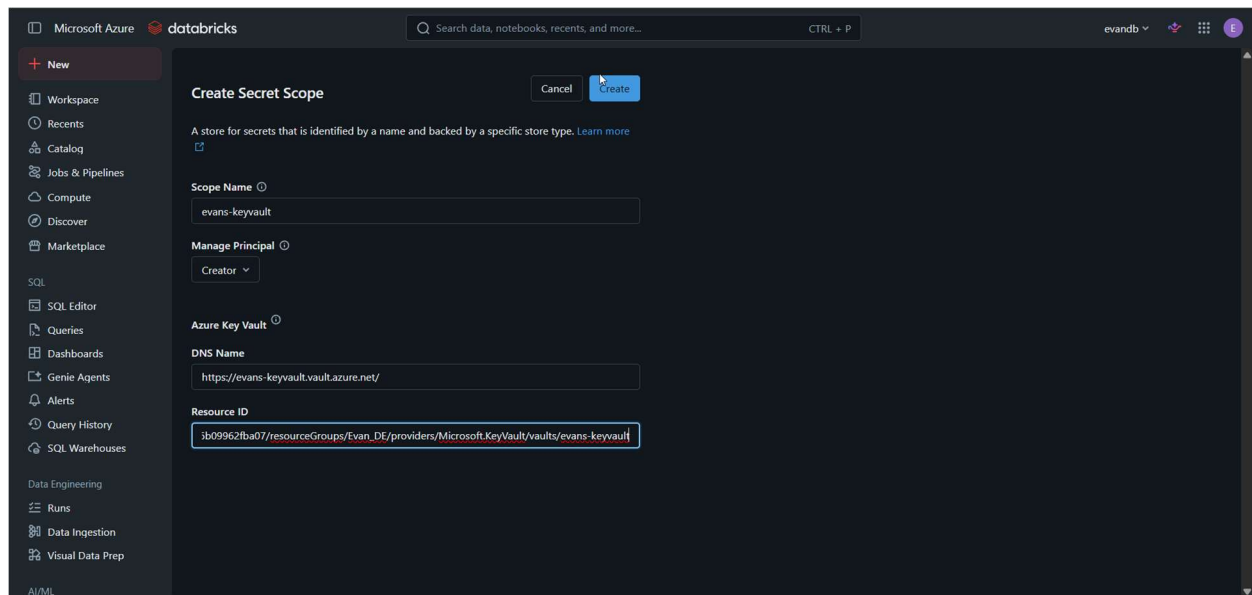
At the bottom of the form, there are two buttons: 'Create' and 'Cancel'. The 'Create' button is highlighted with a tooltip that also says 'Create'.

1. Never include the secret value in GitHub documentation or screenshots.
2. Create/save the secret.
3. Do not use evandb8989 as the secret value. It is only the secret name.
4. In the secret value field, paste the Storage Account Access Key copied from evanpunnen.
5. Set the secret name to: evandb8989.
6. Select Generate/Import (or the equivalent create-secret option shown in your portal).
7. Open Objects → Secrets.
8. Open evans-keyvault.

Reference screenshot: Create a Key Vault secret.

Step 9. Create the Databricks Secret Scope

In Databricks, create a Key Vault-backed secret scope. Use evans-keyvault as the scope name and point it to the Key Vault URI.

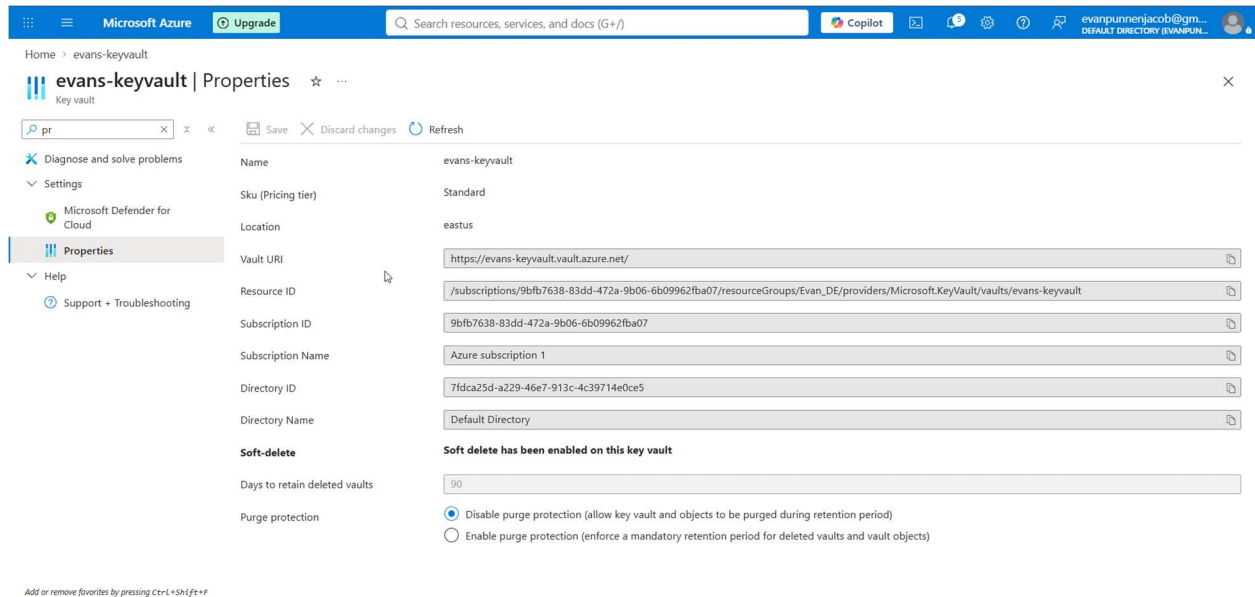


1. The scope name must match the name used later in `dbutils.secrets.get()`.
2. Create the secret scope.
3. Use the appropriate manage principal for your environment. The source screenshot shows Creator.
4. Enter the Key Vault DNS/URI: `https://evans-keyvault.vault.azure.net/`
5. Select Azure Key Vault as the secret scope type.
6. Set the scope name to: `evans-keyvault`.
7. Create a new secret scope.
8. Open the Secret Scope creation page using the Databricks workspace secret-management interface.
9. Return to the Azure Databricks workspace.

Reference screenshot: Key Vault-backed Databricks Secret Scope.

Step 10. Verify Key Vault properties

Confirm the Key Vault name, vault URI, and resource information used by the Databricks secret scope.

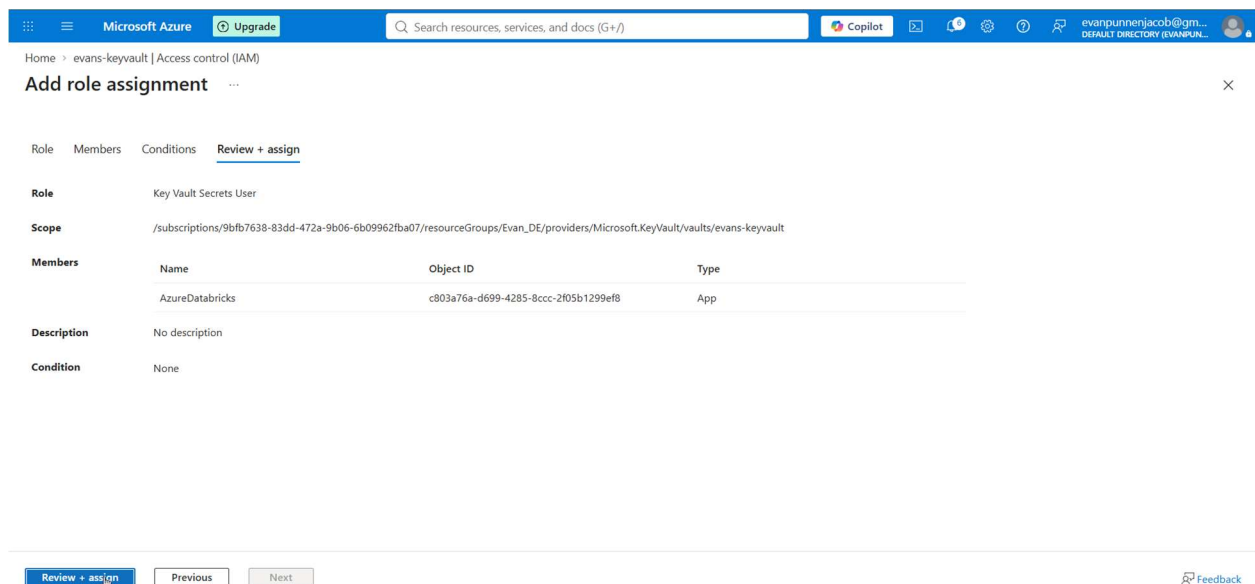


1. Make sure the URI used when creating the Databricks secret scope matches the Key Vault URI.
2. Confirm the Key Vault URI is https://evans-keyvault.vault.azure.net/.
3. Confirm the vault name is evans-keyvault.
4. Open the Key Vault Properties page.
5. Open evans-keyvault in Azure Portal.

Reference screenshot: Key Vault properties.

Step 11. Assign the required Key Vault role to AzureDatabricks

The source tutorial screenshot shows AzureDatabricks assigned the Key Vault Secrets User role at the Key Vault scope. This allows the Databricks identity to retrieve secrets.



1. This allows the Databricks identity to retrieve the secret through the configured Key Vault-backed secret scope.
2. Complete the role assignment.
3. Set the scope to the Key Vault.
4. Assign access to the AzureDatabricks identity used by the workspace.
5. Select the Key Vault Secrets User role.
6. Choose Add role assignment.
7. Select Access control (IAM).
8. Open evans-keyvault in Azure Portal.

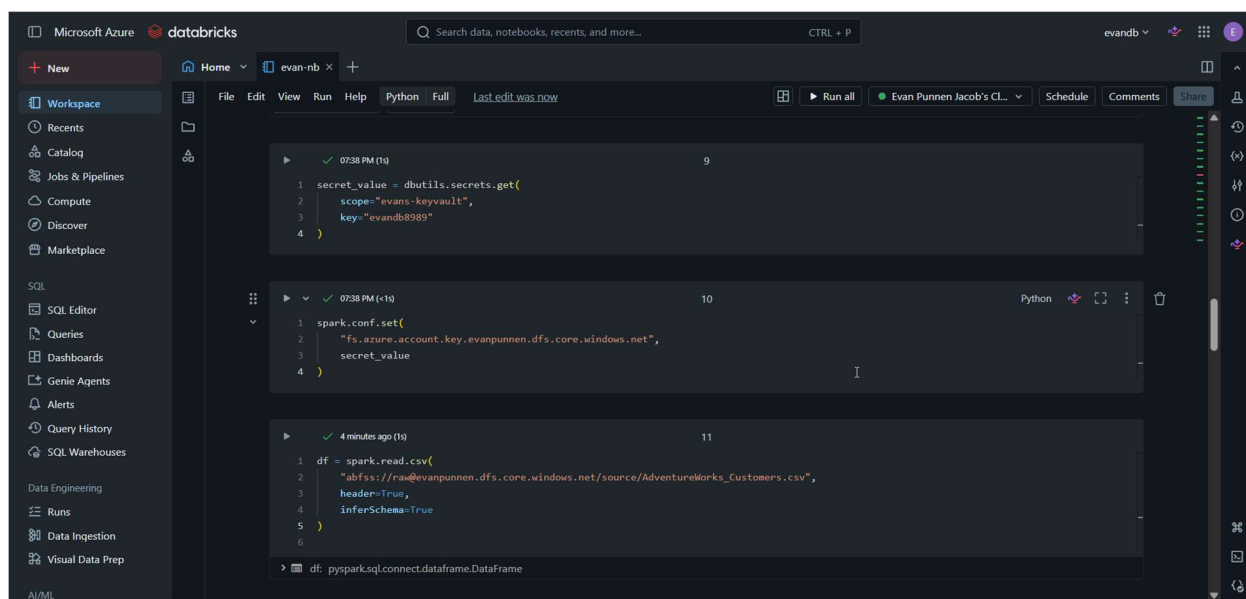
Reference screenshot: AzureDatabricks → Key Vault Secrets User.

Step 12. Retrieve the secret and configure ADLS authentication

In the Python notebook, retrieve the secret through the Databricks Secret Scope and use that value as the ADLS Storage Account Access Key. Do not print or display secret_value.

```
secret_value = dbutils.secrets.get(
    scope="evans-keyvault",
    key="evandb8989"
)

spark.conf.set(
    "fs.azure.account.key.evapunnen.dfs.core.windows.net",
    secret_value
)
```



1. Use the storage account hostname exactly as shown in the code.
2. Do not print secret_value or display it in the notebook output.

3. Configure Spark with the Storage Account Access Key using `spark.conf.set()`.
4. Store the returned value in a Python variable such as `secret_value`.
5. Retrieve the secret from the Databricks secret scope using `dbutils.secrets.get()`.
6. Return to the Python notebook created in Step 1.

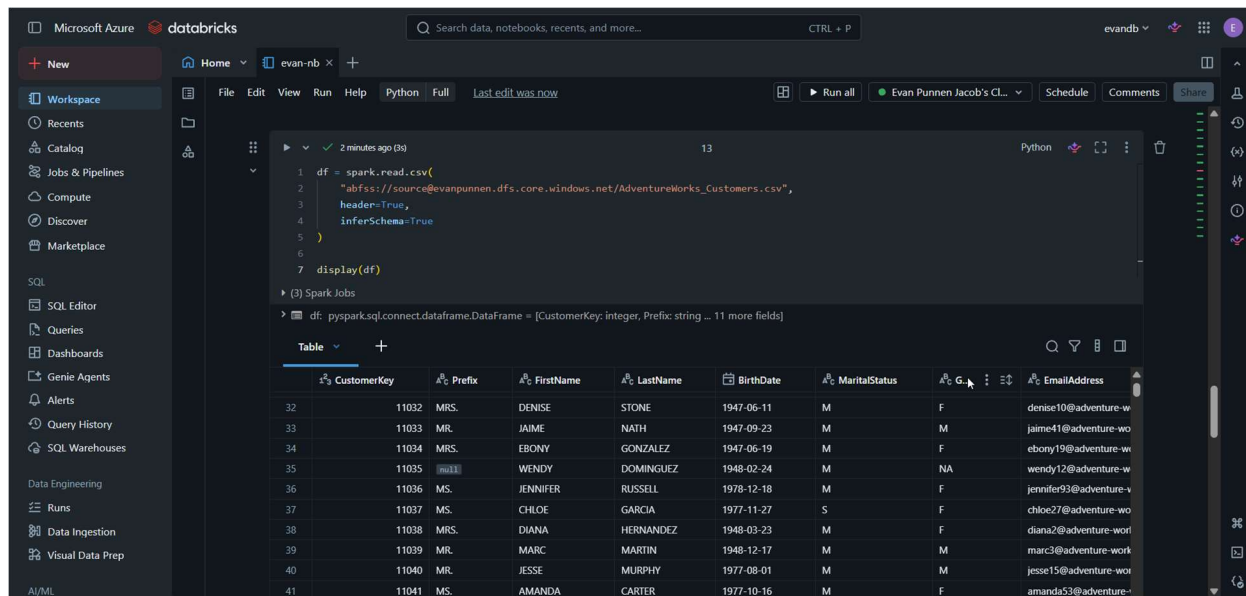
Reference screenshot: Databricks notebook secret retrieval and storage configuration.

Step 13. Read and validate the ADLS Gen2 file

Use the correct ABFSS path. In your environment the container is source and the file is at the container root. The earlier raw/source path was incorrect; the working path is shown below.

```
df = spark.read.csv(
    "abfss://source@evanpunnen.dfs.core.windows.net/AdventureWorks_Customers.csv",
    header=True,
    inferSchema=True
)

display(df)
```



1. If the DataFrame is displayed, the Storage Account → Key Vault → Databricks Secret Scope → ADLS authentication flow has been validated.
2. Use `display(df)` to verify that the DataFrame loads successfully.
3. Run the cell.
4. Read the CSV with `header=True` and `inferSchema=True`.
5. Keep source as the container name; do not add raw/source unless that folder actually exists.
6. Use the ABFSS path shown below.

7. Confirm that AdventureWorks_Customers.csv exists in the source container.

Reference screenshot: correct ABFSS path and DataFrame output.

Final connection flow

Azure Storage (ADLS Gen2) → Storage Account Access Key → Azure Key Vault → Databricks Secret Scope → Spark configuration → ABFSS → PySpark DataFrame